GUÍA COMPLETA

Mantenimiento de una aplicación móvil

Procedimiento práctico desde la planificación hasta la mejora continua

Objetivo	Mantener la aplicación segura, estable, compatible, rápida y útil para sus usuarios.
Ámbito	Aplicaciones Android, iOS o multiplataforma, con backend y servicios asociados cuando existan.
Uso	Lista de trabajo para equipos de desarrollo, QA, operaciones, producto y soporte.

Nota: no todos los pasos aplican a todos los proyectos. Adapta la frecuencia, los responsables y los criterios de aprobación al riesgo, tamaño y arquitectura de tu aplicación.

Índice

- 1. Preparar el mantenimiento
- 2. Revisar incidencias y métricas
- 3. Comprobar código, dependencias y arquitectura
- 4. Seguridad y privacidad
- 5. Pruebas funcionales y técnicas
- 6. Rendimiento, batería y consumo de datos
- 7. Compatibilidad y accesibilidad
- 8. Backend, APIs y servicios externos
- 9. Publicación y despliegue seguro
- 10. Monitorización y respuesta a incidentes
- 11. Soporte, documentación y mejora continua
- 12. Calendario recomendado
- 13. Lista de comprobación antes de publicar
- 14. Indicadores para evaluar el mantenimiento

1. Preparar el mantenimiento

Antes de modificar la aplicación, define el alcance, el riesgo y la forma de comprobar que el trabajo ha terminado.

- Identificar la versión actual publicada, la versión en desarrollo y las plataformas afectadas (Android, iOS, tabletas, etc.).
- Revisar el plan de mantenimiento, los acuerdos de nivel de servicio (si existen), el presupuesto y las ventanas de despliegue.
- Asignar responsables para desarrollo, pruebas, seguridad, infraestructura, producto y soporte.
- Comprobar que existen repositorios, acceso al sistema de incidencias, paneles de analítica, cuentas de publicación y documentación.
- Confirmar copias de seguridad y procedimientos de recuperación para datos y servicios backend.
- Definir criterios de aceptación, plan de pruebas, estrategia de despliegue y procedimiento de reversión.
- Clasificar el trabajo: correctivo (errores), preventivo (evitar fallos), adaptativo (cambios de plataforma) o evolutivo (mejoras).

2. Revisar incidencias y métricas

- Leer los informes de errores, cierres inesperados, bloqueos, reseñas y tickets de soporte.
- Agrupar incidencias por frecuencia, gravedad, versión, dispositivo, sistema operativo y pasos para reproducirlas.
- Priorizar primero los problemas de seguridad, pérdida de datos, indisponibilidad y bloqueos generalizados.
- Comparar las métricas con la versión anterior y con los objetivos acordados.
- Confirmar si los problemas afectan a todos los usuarios o solo a una región, cuenta, dispositivo o flujo concreto.
- Convertir los hallazgos en tareas trazables con responsable, prioridad, fecha objetivo y criterio de resolución.

Métricas útiles: tasa de cierres inesperados, usuarios sin errores, tiempo de inicio, latencia de API, errores de red, conversión de flujos clave, consumo de batería y volumen de incidencias.

3. Comprobar código, dependencias y arquitectura

- Revisar cambios recientes, deuda técnica, duplicaciones, complejidad y módulos con mayor número de defectos.
- Ejecutar análisis estático, reglas de estilo, linters y comprobaciones de calidad configuradas en el proyecto.
- Actualizar gradualmente SDK, bibliotecas, frameworks, plugins y herramientas de compilación; leer notas de versión y cambios incompatibles.
- Eliminar dependencias sin uso y verificar que las bibliotecas siguen mantenidas y tienen un origen fiable.
- Revisar permisos solicitados, configuración por entorno, gestión de errores y tratamiento de estados sin conexión.
- Comprobar que secretos, claves privadas, tokens y credenciales no estén incluidos en el código ni en el repositorio.
- Mantener ramas, revisiones de código y registros de cambios; evitar modificaciones directas en producción.

- Verificar que la compilación sea reproducible y que el proceso de integración continua funcione.

4. Seguridad y privacidad

- Analizar vulnerabilidades conocidas en dependencias y componentes propios; corregir primero las críticas y altas según el riesgo.
- Aplicar autenticación robusta, autorización en servidor, gestión segura de sesiones y cierre de sesión efectivo.
- Guardar credenciales y tokens con los mecanismos seguros de la plataforma; no almacenarlos en texto plano.
- Usar TLS para las comunicaciones y validar correctamente certificados y respuestas del servidor.
- Minimizar los datos personales recopilados; documentar finalidad, conservación, acceso y eliminación.
- Revisar permisos del dispositivo para pedir solo los necesarios y explicarlos en el momento adecuado.
- Comprobar que los registros y herramientas analíticas no incluyan contraseñas, tokens ni datos personales innecesarios.
- Revisar consentimiento, política de privacidad, gestión de solicitudes de derechos y obligaciones legales aplicables.
- Probar controles frente a abuso, manipulación de parámetros, acceso indebido y exposición de datos.
- Definir cómo revocar claves, rotar credenciales, notificar incidentes y distribuir una corrección urgente.

La seguridad no se resuelve con una única auditoría: debe incorporarse al desarrollo, las pruebas, la publicación y la monitorización.

5. Pruebas funcionales y técnicas

5.1. Pruebas funcionales

- Verificar inicio y cierre de sesión, recuperación de cuenta y permisos por tipo de usuario.
- Probar los flujos principales de negocio, formularios, búsquedas, pagos o compras (si existen), notificaciones y ajustes.
- Comprobar validaciones, mensajes de error, estados vacíos, carga, reintentos y recuperación tras perder la conexión.
- Confirmar que los datos se guardan, sincronizan y muestran correctamente, incluidos los casos límite.

5.2. Pruebas técnicas

- Ejecutar pruebas unitarias, de integración, de interfaz y de regresión.
- Probar instalaciones nuevas, actualización desde versiones anteriores y migraciones de datos locales.
- Verificar escenarios de permisos denegados, poco almacenamiento, interrupciones, modo avión y procesos en segundo plano.
- Realizar pruebas exploratorias en dispositivos reales además de simuladores o emuladores.
- Comprobar que las pruebas automatizadas sean estables y que los fallos no se ignoren sin justificación.
- Registrar resultados, evidencias, defectos detectados y cobertura de los flujos críticos.

6. Rendimiento, batería y consumo de datos

- Medir tiempo de arranque, fluidez, memoria, CPU, tamaño de la aplicación y latencia de red.
- Buscar bloqueos de interfaz, fugas de memoria, tareas pesadas en el hilo principal y operaciones redundantes.

- Optimizar imágenes, vídeo, caché, paginación y carga diferida cuando corresponda.
- Reducir peticiones innecesarias, gestionar reintentos con límites y evitar bucles de solicitudes.
- Revisar uso de GPS, sensores, sincronización y tareas en segundo plano para controlar la batería.
- Probar con redes lentas o inestables y definir límites de tiempo de espera y mensajes adecuados.
- Comparar resultados con una línea base para confirmar que las optimizaciones mejoran el comportamiento real.

7. Compatibilidad y accesibilidad

- Probar las versiones de Android e iOS que el producto declara compatibles, incluidos los cambios recientes del sistema.
- Cubrir una matriz representativa de dispositivos, tamaños de pantalla, resoluciones, fabricantes y niveles de rendimiento.
- Verificar orientación, teclado, áreas seguras, notificaciones, enlaces profundos y comportamiento al volver a la aplicación.
- Comprobar escalado de texto, contraste, etiquetas para lectores de pantalla, foco, controles táctiles y navegación accesible.
- Probar idiomas, formatos de fecha, hora, números, moneda y zonas horarias que soporte el producto.
- Revisar los requisitos actuales de las tiendas y los SDK objetivo antes de cada publicación.

8. Backend, APIs y servicios externos

- Comprobar disponibilidad, latencia, tasa de error y límites de uso de las APIs.
- Verificar compatibilidad entre la versión móvil publicada y los cambios del backend.
- Mantener contratos de API documentados y evitar cambios incompatibles sin un plan de transición.
- Probar caducidad de sesión, respuestas incompletas, errores del servidor y límites de tasa.
- Revisar bases de datos, colas, tareas programadas, almacenamiento, certificados y dominios.
- Confirmar que los proveedores externos (pagos, mapas, analítica, notificaciones, autenticación) siguen funcionando.
- Preparar degradación controlada o alternativas cuando un servicio externo esté caído.

9. Publicación y despliegue seguro

- Crear una versión identificable y revisar número de versión, notas de publicación, permisos y configuración.
- Ejecutar la batería de pruebas en un entorno de preproducción equivalente a producción.
- Generar artefactos firmados de forma segura y limitar el acceso a certificados y credenciales de publicación.
- Verificar que no se distribuyan endpoints de pruebas, datos ficticios, depuración activa ni secretos.
- Publicar primero a un grupo interno o una fracción de usuarios cuando la plataforma lo permita.
- Observar métricas y errores durante el despliegue; ampliar progresivamente si no aparecen señales negativas.
- Tener preparado un mecanismo de desactivación de funciones, reversión del backend o corrección urgente.
- Confirmar la instalación, actualización y funcionamiento desde las tiendas oficiales.
- Documentar la versión, cambios, incidencias conocidas, responsable y decisión final de publicación.

10. Monitorización y respuesta a incidentes

- Configurar alertas accionables para caídas, aumentos de errores, latencia, fallos de autenticación y saturación.
- Verificar que la monitorización no recopile más datos personales de los necesarios.
- Establecer niveles de severidad, canal de comunicación, responsable de guardia y criterios de escalado.
- Ante un incidente: confirmar alcance, proteger a usuarios y datos, mitigar el impacto y comunicar el estado.
- Conservar evidencias útiles sin exponer información sensible y registrar una línea temporal de acciones.
- Después de resolverlo, realizar análisis de causa raíz, crear acciones preventivas y comprobar su cumplimiento.
- Probar periódicamente restauración de copias de seguridad y procedimientos de recuperación.

11. Soporte, documentación y mejora continua

- Mantener una base de conocimiento con problemas frecuentes, soluciones y pasos de diagnóstico.
- Responder a usuarios con información clara, sin prometer fechas que no estén confirmadas.
- Actualizar diagramas de arquitectura, contratos de API, configuración, dependencias y procedimientos operativos.
- Registrar decisiones técnicas relevantes y cambios que afecten a compatibilidad o datos.
- Revisar solicitudes de funciones y comentarios; distinguir problemas recurrentes de casos aislados.
- Eliminar funciones obsoletas de forma planificada y medir el impacto antes de retirarlas.
- Programar retrospectivas y convertir las lecciones aprendidas en mejoras concretas.

12. Calendario recomendado

La frecuencia depende del riesgo y del ritmo de cambios. Este calendario es una referencia inicial, no una obligación universal.

Frecuencia	Actividades
Diaria o continua	Monitorizar disponibilidad, errores, cierres inesperados, alertas de seguridad e incidencias urgentes.
Semanal	Revisar tickets, tendencias, copias de seguridad, pipeline CI/CD, dependencias vulnerables y tareas prioritarias.
Mensual	Revisar rendimiento, compatibilidad, permisos, dependencias, costes de servicios y calidad de soporte.
Trimestral	Revisar arquitectura, deuda técnica, accesibilidad, recuperación ante desastres, privacidad y objetivos de producto.
Antes de cada versión	Completar pruebas de regresión, revisión de seguridad, notas de versión, plan de despliegue y reversión.
Ante una alerta crítica	Activar el procedimiento de incidentes, mitigar, informar, corregir y documentar el aprendizaje.

13. Lista de comprobación antes de publicar

Marca cada punto antes de autorizar una versión. Si un punto no aplica, deja constancia de la razón.

☐ La incidencia o mejora está descrita y tiene criterios de aceptación claros.

☐ El código ha sido revisado y los análisis automáticos han finalizado correctamente.

☐ No quedan vulnerabilidades críticas o altas sin una aceptación de riesgo documentada.

☐ Las pruebas unitarias, integración, interfaz y regresión relevantes han pasado.

☐ Se han probado los flujos críticos en dispositivos y versiones de sistema representativos.

☐ El rendimiento no ha empeorado de forma relevante respecto a la línea base.

☐ La privacidad, permisos, textos y datos analíticos se han revisado.

☐ La configuración apunta al entorno correcto y no contiene secretos ni datos de prueba.

☐ Las migraciones y cambios de API son compatibles o tienen un plan de transición.

☐ Las notas de versión, soporte y documentación están actualizadas.

☐ Existe un plan de despliegue, monitorización y reversión, con responsables asignados.

☐ El despliegue gradual se ha validado y se ha definido cuándo detenerlo.

14. Indicadores para evaluar el mantenimiento

Indicador	Qué muestra	Uso recomendado
Tasa de cierres inesperados	Estabilidad de la aplicación.	Seguir tendencia por versión y dispositivo.
Usuarios sin errores	Proporción de usuarios que no experimentan fallos.	Medir antes y después de cambios.
Tiempo de inicio	Rapidez de apertura.	Definir percentiles y dispositivos de referencia.
Latencia y errores de API	Salud de la comunicación con servicios.	Separar errores de cliente y servidor.
Tiempo de resolución	Velocidad para corregir incidencias.	Separar por severidad.
Tasa de regresiones	Problemas introducidos por cambios.	Revisar pruebas y proceso de publicación.
Adopción de versiones	Rapidez con la que los usuarios actualizan.	Detectar versiones antiguas y riesgos.

Anexo: registro mínimo de mantenimiento

Para cada intervención, registra: fecha, versión afectada, motivo, prioridad, responsable, cambios realizados, pruebas ejecutadas, resultado, riesgos conocidos, aprobación, hora de despliegue y plan de reversión. Esto facilita auditorías, análisis de incidentes y continuidad del equipo.

Principio final: mantener una aplicación móvil es un proceso continuo. Prioriza la seguridad y los datos de los usuarios, valida cada cambio con pruebas y usa métricas reales para decidir qué mejorar.